BACKEND DEVELOPER STUDY GUIDE

Authentication, Session Authentication, and JWT

Backend developer study notes

Concepts, Django and DRF request flows, browser security, implementation tradeoffs, and high-level interview review

Updated: October 2026

AUTHN SESSIONS JWT BROWSER SECURITY

How to use these notes

These notes are organized around one mental model: credentials establish an identity, an authentication mechanism carries that identity across requests, and authorization decides what that identity may do.

For interviews, do not only describe how to configure a library. Explain the threat being addressed, the trust boundary, the failure mode, and the tradeoff introduced by the design.

Key questions to ask about any authentication design:

- What credential proves the user's identity?
- Where is authenticated state stored?
- How is the credential transported?
- What can steal or replay it?
- How does it expire or get revoked?
- Where and how is authorization enforced?

1. Core terminology

Authentication and authorization

Authentication, often abbreviated AuthN, answers: Who is making this request?

Authorization, often abbreviated AuthZ, answers: Is this authenticated principal allowed to perform this action on this resource?

Example:

```
Session or JWT identifies User 17 -> authentication
User 17 has the vendor role -> role authorization
Product 42 belongs to User 17's vendor -> object authorization
Vendor may update but not approve the product -> action authorization
```

Authentication must happen before authorization, but a successfully authenticated user is not automatically authorized.

Important terms

- Identity: an account or entity, such as User(id=17).
- Principal: the entity currently associated with a request.
- Credential: evidence used to prove identity, such as a password or private key.
- Authenticator: a password, OTP, passkey, certificate, or security key.
- Session: continuing authenticated state across multiple requests.
- Token: a value representing authentication or delegated authorization.
- Bearer token: possession is sufficient to use it; whoever bears it can act with its authority.
- Claim: a statement inside a token, such as subject, issuer, audience, or expiration.
- Scope: a delegated capability, such as orders:read.
- Role: a collection or category of permissions, such as vendor or admin.

401 versus 403

- 401 Unauthorized means authentication credentials are absent, invalid, or expired.
- 403 Forbidden means the caller is known but lacks permission.

- 404 Not Found may deliberately hide the existence of a protected object.

Despite its historical name, HTTP 401 represents an authentication problem, not an authorization decision.

2. Password authentication fundamentals

Passwords must never be stored as plaintext or with reversible encryption. Store a deliberately slow password hash.

Suitable password-hashing families include Argon2id, scrypt, bcrypt, and PBKDF2 where compatibility requirements apply.

Salt, pepper, and work factor

- Salt: a unique random value for each password hash. It prevents identical passwords from producing identical stored hashes and defeats precomputed tables. The salt is normally stored with the hash.
- Pepper: an optional application secret stored separately from the password database.
- Work factor: makes each password guess intentionally expensive. It should be increased over time as hardware improves.

Login endpoint protections

- Rate-limit attempts by account and by network signals.
- Use generic failure messages to reduce username enumeration.
- Compare secrets using constant-time operations where applicable.
- Record security events without logging passwords or tokens.
- Add MFA or passkeys for higher-risk accounts.
- Re-authenticate users before sensitive operations.
- Treat password-reset flows as authentication flows.

Password-reset tokens should be random, short-lived, single-use, and invalidated after success. Password changes may also invalidate existing sessions or refresh-token families.

3. Session authentication

Session authentication stores authenticated state on the server and gives the browser a random session identifier.

```
1. Browser submits username and password
2. Server verifies the password
3. Server creates a session record
4. Server returns a sessionid cookie
5. Browser automatically includes the cookie on later requests
6. Server uses the session ID to load the user
```

The cookie should contain an unpredictable identifier rather than the user's password or full user record.

Django session request flow

```
HTTP request with sessionid cookie
|
v
SessionMiddleware loads request.session
|
v
```

```
AuthenticationMiddleware resolves request.user
|
v
DRF SessionAuthentication accepts that user
|
v
Permission classes authorize the operation
```

In your Django project, the important middleware ordering is:

```
MIDDLEWARE = [
# ...
'django.contrib.sessions.middleware.SessionMiddleware',
'django.middleware.csrf.CsrfViewMiddleware',
'django.contrib.auth.middleware.AuthenticationMiddleware',
# ...
]
```

SessionMiddleware must run before AuthenticationMiddleware because authentication reads the session.

DRF configuration:

```
REST_FRAMEWORK = {
'DEFAULT_AUTHENTICATION_CLASSES': [
'rest_framework.authentication.SessionAuthentication',
],
}
```

Login

```
user = authenticate(
request,
username=username,
password=password,
)

if user is None:
return JsonResponse({'detail': 'Invalid credentials'}, status=401)

django_login(request, user)
```

authenticate verifies the credentials. django_login associates the user with a newly created or rotated server-side session and causes a session cookie to be returned.

Page refresh with a session

React state is memory and disappears on refresh. The browser cookie and the server-side session normally remain.

```
Page refresh
-> React starts with user = undefined
-> frontend calls GET /api/auth/me/
-> browser automatically sends sessionid
-> Django reconstructs request.user
-> endpoint returns public user data
-> React calls setUser(user)
-> cart and orders are fetched again
```

Use three frontend states:

```
undefined -> authentication check is still running
null -> no authenticated user
object -> authenticated user information
```

Logout

Server-side logout invalidates or flushes the session and expires the cookie. The frontend should also clear user-specific state such as cached cart and order data.

Session strengths and costs

Strengths:

- Immediate server-side revocation is straightforward.
- Browser security attributes work naturally with cookies.
- The session can reflect current server state.
- It is simple for a first-party Django web application.

Costs:

- The server needs a session lookup.
- Multiple application instances need a shared session store, such as Redis or a database.
- Automatically sent cookies require CSRF defenses for unsafe operations.

Session authentication is not inherently obsolete or unscalable. A shared session store is a normal distributed-system design.

4. Cookie security and CSRF

A production session cookie should generally use:

```
HttpOnly -> JavaScript cannot read the cookie value
Secure -> cookie is sent only over HTTPS
SameSite -> limits cross-site cookie sending
Path -> restricts where the cookie is sent
Domain -> should be as narrow as possible
```

Example Django production settings:

```
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SECURE = True
SESSION_COOKIE_SAMESITE = 'Lax'

CSRF_COOKIE_SECURE = True
CSRF_COOKIE_SAMESITE = 'Lax'
```

Why session ID and CSRF token are both needed

- The session ID answers: Which authenticated session is making this request?
- The CSRF token answers: Was this unsafe request intentionally created by the trusted frontend?

The browser automatically attaches cookies. A malicious site may therefore cause a victim's browser to send an authenticated request. Django requires the submitted CSRF token to match the expected token.

Typical unsafe request:

```
POST /api/orders/
Cookie: sessionid=abc123; csrftoken=xyz789
X-CSRFToken: xyz789
```

HttpOnly prevents JavaScript from extracting the session ID, but it does not make XSS harmless. Malicious JavaScript running inside the trusted origin may still perform actions while it is active.

SameSite is defense in depth and should not automatically be treated as a replacement for framework CSRF protection.

5. CORS is not authentication

CORS is a browser-enforced response-sharing policy. It decides whether JavaScript from one origin may read responses from another origin.

CORS does not:

- authenticate a user;
- authorize access;
- prevent curl, Postman, mobile applications, or servers from calling an API;
- replace CSRF protection;
- secure an endpoint whose backend permissions are missing.

Strong interview statement:

```
CORS is a browser policy, not an API security boundary.
```

6. What a JWT is

JWT means JSON Web Token. It is a token format, not a complete authentication protocol.

A compact signed JWT contains three segments:

```
base64url(header).base64url(payload).base64url(signature)
```

Example structure:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.
eyJ1c2VyX2lkIjoxNywidG9rZW5fdHlwZSI6ImFjY2VzcyJ9
.
signature-bytes-encoded-for-transport
```

JWT header

```
{
"alg": "HS256",
"typ": "JWT"
}
```

The header is untrusted token metadata. It may describe an algorithm, type, and key identifier. The server must compare it with trusted server-side configuration rather than allowing the incoming token to choose arbitrary security behavior.

JWT payload

```
{
"user_id": 17,
"token_type": "access",
"iss": "https://auth.example.com",
"aud": "ecommerce-api",
"iat": 1791012045,
"exp": 1791012345,
"jti": "unique-token-id"
}
```

The payload contains claims. It does not normally contain the complete database user.

JWT signature

The signature protects the encoded header and encoded payload against undetected modification.

```
signing_input = encoded_header + "." + encoded_payload
signature = sign(signing_input, signing_key)
```

7. Encoding, signing, and encryption

These operations solve different problems.

- Encoding makes bytes safe to transport. It is reversible without a secret.
- Signing detects modification and proves possession of a signing key.
- Encryption hides content from parties without the decryption key.

A normal signed JWT is Base64URL-encoded and signed. It is usually not encrypted.

Anyone who receives a standard JWT can decode its header and payload. Never place passwords, card data, secret keys, or confidential personal information in an ordinary JWT payload.

Base64URL uses URL-safe characters and commonly omits padding. It provides no confidentiality.

8. How JWT signature validation works

HS256: symmetric signing

With HS256, issuer and verifier share the same secret.

```
expected_signature = HMAC-SHA256(
encoded_header + "." + encoded_payload,
shared_secret
)
```

The verifier recalculates the expected signature and performs a constant-time comparison with the signature received in the JWT.

If an attacker changes user_id or role, the signing input changes. Without the shared secret, the attacker cannot create the new required signature.

Important tradeoff: every service possessing an HMAC verification secret can also mint valid tokens.

RS256 or ES256: asymmetric signing

```
Authorization server: private key signs
Resource servers: public key verifies
```

A service with only the public key can validate tokens but cannot create new valid tokens. This separation is useful when multiple independent services validate tokens.

Key identifiers and rotation

A JWT header may contain kid, a key identifier. The verifier can use it to select a trusted key from a controlled key set such as JWKS.

kid is not itself trusted. It must never be used unsafely as a file path, command fragment, or database query. Key rotation should allow a transition period where currently valid tokens can still be verified with an older trusted public key.

9. Complete JWT validation

A valid signature is necessary but not sufficient.

Recommended validation sequence:

```
1. Extract token from an approved location
2. Require the expected token format
3. Enforce an algorithm allowlist
4. Select a trusted verification key
5. Verify the signature
6. Validate exp and nbf with a small clock-skew allowance
7. Validate expected issuer, iss
8. Validate expected audience, aud
9. Validate token type and required claims
10. Resolve or construct the principal
11. Apply authorization and object-level permissions
```

Common claims:

- sub: stable subject identifier.
- iss: trusted issuer.
- aud: intended API or resource server.
- exp: expiration time.
- nbf: token must not be accepted before this time.
- iat: issuance time.
- jti: unique token identifier; useful for audit or revocation state.

Examples of rejected tokens:

```
Valid signature + expired exp -> reject
Valid signature + wrong audience -> reject
Valid signature + untrusted issuer -> reject
```

```
Valid refresh token used as access token -> reject
Valid token + disabled user -> reject according to policy
```

The server must never blindly trust alg from the token header. Configure acceptable algorithms on the verifier.

10. JWT authentication in Django REST Framework

Typical Simple JWT configuration:

```
REST_FRAMEWORK = {
'DEFAULT_AUTHENTICATION_CLASSES': [
'rest_framework_simplejwt.authentication.JWTAuthentication',
],
}
```

Typical authenticated request:

```
GET /api/orders/
Authorization: Bearer eyJhbGciOi...
```

The HTTP Authorization header transports the JWT. It does not itself validate anything.

Do not confuse the two headers:

- HTTP Authorization header: carries Bearer and the compact token.
- Internal JWT header: contains metadata such as alg, typ, and possibly kid.

DRF flow:

```
Read HTTP Authorization header
-> require Bearer scheme
-> extract compact JWT
-> validate signature and claims
-> read user_id
-> load the current User
-> set request.user
-> run permission classes
```

Normal Simple JWT authentication can validate the signature without a session lookup, but it generally performs a database lookup to obtain request.user.

```
No session lookup does not mean no database lookup.
```

A stateless token-user implementation can avoid loading the database User, but it may not suit applications that need current roles, vendor profiles, account status, or object ownership.

11. Access and refresh tokens

Access token

- Short-lived, often minutes.
- Sent to resource APIs.
- Carries limited authority and audience.

- Used in Authorization: Bearer.

Refresh token

- Longer-lived and more sensitive.
- Sent only to the authorization server or refresh endpoint.
- Used to obtain new access tokens.
- Should support expiration, revocation, and rotation.
- Should never be sent with every normal API request.

Typical flow:

```
Login credentials
-> access token + refresh token
-> access token calls APIs
-> access token expires
-> refresh token obtains a new access token
-> logout revokes or blacklists the refresh token
```

Refresh-token rotation

```
Client submits refresh token A
-> server invalidates A
-> server returns access token plus refresh token B
```

If both an attacker and the legitimate client use A, reuse of the already consumed token can signal theft. The server may revoke the whole token family and require login again.

12. JWT logout and revocation

A purely stateless access token remains independently valid until expiration. Logout therefore differs from deleting a server session.

Common strategies:

- Keep access tokens short-lived.
- Revoke or blacklist refresh tokens.
- Denylist access-token jti values when immediate revocation is essential.
- Store a per-user token version or valid-after timestamp.
- Revoke token families after password changes or suspected compromise.
- Use token introspection when centralized control is more important than stateless validation.

Tradeoff:

```
Pure stateless validation -> fewer central lookups, harder immediate revocation
Revocation lookup -> stronger control, less stateless
```

Deleting a token from one browser does not invalidate a stolen copy. Server-side revocation policy is still needed for important threats.

13. JWT claims and stale authorization data

Claims are snapshots made when the token is issued.

Suppose a token says:

```
{
"user_id": 17,
"role": "vendor"
}
```

If the database role later becomes customer, the existing token may continue saying vendor until it expires.

Ways to manage staleness:

- Use short access-token lifetimes.
- Load current authorization data from the database.
- Use a token or permission version.
- Revoke existing token families.
- Avoid embedding fast-changing resource permissions.

For an e-commerce API, product ownership, current inventory permissions, account status, and tenant boundaries should generally be checked against current server data.

14. Frontend JWT flow

Login

```
1. Submit username and password to /api/auth/login/
2. Receive access and refresh tokens
3. Store tokens according to the selected security architecture
4. Call /api/auth/me/ using the access token
5. Store public user information in React state
6. Load cart and order data
```

Axios request interceptor

```
api.interceptors.request.use((config) => {
const accessToken = getAccessToken();

if (accessToken) {
config.headers.Authorization = `Bearer ${accessToken}`;
}

return config;
});
```

The interceptor centralizes the Authorization header so each component does not implement token attachment separately.

Expired access token

```
API request returns 401
-> response interceptor checks whether retry is allowed
-> one refresh request is started
-> new access token is stored
-> original request is retried once
```

Important safeguards:

- Mark the original request so it is retried only once.
- Do not recursively intercept the refresh request.
- Coordinate simultaneous failures so they share one refresh operation.
- If refresh fails, clear local authentication state and return to login.

Page refresh with JWT

If an access token is stored persistently, the app calls /auth/me/ and sends it. If only the refresh token persists, the app first refreshes an access token and then calls /auth/me/.

React's user state is still only presentation state. The backend token and permissions remain the real security boundary.

Logout

```
Send refresh token to revocation/blacklist endpoint
-> clear access and refresh credentials locally
-> clear user, cart, and orders from React state
-> navigate to login
```

15. Browser token storage

localStorage and sessionStorage

JavaScript executing in the origin can read both. A single XSS vulnerability can exfiltrate stored bearer tokens.

sessionStorage reduces persistence to a tab lifetime but does not prevent XSS theft.

HttpOnly cookies

JavaScript cannot read an HttpOnly cookie. Secure and SameSite should also be configured appropriately.

Because browsers automatically attach cookies, cookie-carried authentication requires CSRF analysis and usually CSRF protection on unsafe endpoints.

Common architecture choices

First-party Django web application:

```
Django session in Secure, HttpOnly cookie
+ CSRF protection
```

JWT browser application requiring JWT semantics:

```
Short-lived access token in memory
+ refresh token in Secure, HttpOnly cookie
+ CSRF protection for refresh and logout endpoints
```

Backend-for-frontend pattern:

```
Browser holds only a hardened session cookie
BFF stores or manages downstream OAuth tokens
```

Avoid persistent authentication tokens in Web Storage for production systems when a safer design is practical.

16. Sessions versus JWT

Prefer sessions when

- The application is a first-party browser frontend plus one backend.
- Immediate logout and revocation are important.
- Centralized session state is acceptable.
- Simplicity and secure browser-cookie integration are priorities.

Consider JWT when

- Multiple resource servers need independently verifiable tokens.
- Mobile, machine, or third-party clients need API credentials.
- OAuth or OpenID Connect integration is required.
- Audience-restricted delegated access is needed.

Avoid weak reasoning

Weak statement:

```
JWT is always more scalable and modern.
```

Stronger statement:

```
JWT moves some authentication state into signed claims and enables local
verification, but introduces revocation, token-storage, claim-staleness,
and key-management tradeoffs. Sessions offer centralized control and are
often the simpler design for a first-party web application.
```

17. Authorization models

RBAC: role-based access control

```
vendor -> create and update owned products
admin -> manage users and approve products
```

ABAC: attribute-based access control

```
Allow update when:
user.role == vendor
and product.vendor_id == user.vendor_id
and product.status != locked
```

ReBAC: relationship-based access control

```
User may view an order because they own it.
Manager may view it because they manage the owner.
```

Object-level checks prevent IDOR or BOLA vulnerabilities. IsAuthenticated alone does not stop User A from requesting User B's object if the queryset or object permission is incorrect.

In DRF, restrict both the queryset and object mutation:

```
def get_queryset(self):
return Order.objects.filter(user=self.request.user)
```

```
def has_object_permission(self, request, view, obj):
return obj.user_id == request.user.id
```

18. OAuth 2.0 and OpenID Connect

- JWT is a token format.
- OAuth 2.0 is a delegated authorization framework.
- OpenID Connect is an identity layer built on OAuth 2.0.

Token purposes:

- Access token: presented to a resource API.
- ID token: tells the client about an authenticated user.
- Refresh token: obtains new access tokens.

Do not use an ID token as a general API access token. The intended audience and validation rules differ.

Authorization Code Flow with PKCE

```
Client generates random code_verifier
-> derives code_challenge
-> authorization request carries challenge
-> client receives authorization code
-> token request carries code plus verifier
-> server verifies the relationship
-> tokens are issued
```

PKCE protects an intercepted authorization code from redemption by a party that lacks the verifier.

A browser SPA is a public client. A client secret embedded in JavaScript is not confidential and must not be treated as proof of the SPA's identity.

19. High-level interview questions

Is a JWT encrypted?

Usually no. A standard signed JWT is readable but tamper-evident. Encryption is a separate operation.

Is only the signature checked?

No. Verify the allowed algorithm and signature, then validate time, issuer, audience, token type, and other required claims before authorization.

Is a JWT validated through the HTTP header?

The HTTP Authorization header transports it. The verifier validates the JWT's signature and claims. The internal JWT header is token metadata, not trusted server configuration.

Can JWT validation happen without a database?

Cryptographic validation can. The application may still load a user, permissions, revocation status, or resource ownership from a database.

Can a user modify a JWT?

Anyone can edit the encoded bytes, but modification invalidates the signature unless they possess the signing key or exploit a verifier flaw.

How do you revoke a JWT immediately?

Introduce state, such as a denylist, token version, valid-after timestamp, or introspection. Otherwise, a valid access token lives until expiration.

Why validate audience?

To prevent a valid token intended for Service A from being accepted by Service B.

Should roles be placed in a JWT?

Stable, coarsely scoped roles may be included, but claims can become stale. Sensitive or fast-changing permissions should be checked against current server state.

Does JWT eliminate CSRF?

Not inherently. An Authorization header manually added by application code reduces classic CSRF exposure. A JWT automatically sent in a cookie still requires CSRF defenses.

Does HttpOnly eliminate XSS?

No. It prevents scripts from reading that cookie, but malicious same-origin JavaScript may still make authenticated requests while running.

JWT or session?

Choose based on clients, trust boundaries, revocation requirements, distribution, and operational complexity. Neither is universally superior.

20. Security and design checklist

Authentication:

- Use established libraries and framework primitives.
- Hash passwords with a suitable adaptive algorithm.
- Rate-limit authentication and recovery endpoints.
- Support MFA for sensitive accounts.
- Avoid account enumeration and secret logging.

Sessions and cookies:

- Use high-entropy identifiers.
- Rotate identifiers after authentication or privilege changes.
- Set HttpOnly, Secure, and appropriate SameSite attributes.
- Enforce CSRF protection for unsafe cookie-authenticated requests.
- Provide expiration, logout, and server-side revocation.

JWT:

- Enforce an algorithm allowlist.
- Protect signing keys and plan rotation.
- Validate signature, exp, nbf, iss, aud, and token type.
- Keep access tokens short-lived and narrowly scoped.
- Protect and rotate refresh tokens.
- Do not place confidential information in readable claims.
- Design explicitly for logout, password change, and account compromise.

Authorization:

- Enforce permissions on the backend.
- Restrict querysets by owner or tenant.
- Check object-level permissions.
- Treat token claims as potentially stale.
- Test horizontal and vertical privilege escalation.

Operations:

- Use HTTPS end to end.
- Keep tokens out of logs, URLs, analytics, and error reports.
- Synchronize clocks and use only small leeway.
- Audit security-relevant events.
- Monitor credential stuffing, token replay, and unusual refresh activity.
- Fail closed when validation is incomplete or ambiguous.

21. Final mental model

```
Credentials establish identity.
Sessions or tokens carry authenticated identity between requests.
Signatures protect token integrity, not secrecy.
Claims must be validated, not merely decoded.
Authorization is always a server-side decision.
Browser storage determines which attack can steal or replay credentials.
Expiration limits damage; revocation provides control.
The best design follows the system's actual trust boundaries.
```

References

- OWASP Authentication Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP Password Storage Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP Session Management Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- OWASP JSON Web Token Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_Cheat_Sheet.html
- RFC 7519, JSON Web Token: https://www.rfc-editor.org/rfc/rfc7519
- RFC 8725, JWT Best Current Practices: https://www.rfc-editor.org/rfc/rfc8725
- RFC 9700, OAuth 2.0 Security Best Current Practice: https://www.rfc-editor.org/rfc/rfc9700

- OpenID Connect Core: https://openid.net/specs/openid-connect-core-1_0.html
- Django authentication documentation: https://docs.djangoproject.com/en/stable/topics/auth/
- Django REST Framework authentication: https://www.django-rest-framework.org/api-guide/authentication/
- Simple JWT documentation: https://django-rest-framework-simplejwt.readthedocs.io/